

Guaranteed-Friend Lunch Seating: Implementation Report for the Simulation and Solver

Isaac Lin*

September 2026

Abstract

This report documents the code behind the guaranteed-friend lunch seating demonstration as it is after the September 2026 audit: the synthetic cohort generator, the submission rules, the coercion and demand screens run on every rotation state's effective lists, the feasibility certificate, the four-stage seating solver (pod construction, repair, closure-move annealing, hinted CP-SAT) with its violation-monotonicity invariant, the random baseline and its closed form, the chart-only leakage probe, the statistics, and the validation, determinism and timing facts. Every number here is read from the committed trace files by `paper/make_tables.py`. In the honest scenario every submitter sits with a listed friend in all 16 rotations, 99.68% of submitters sit with exactly one listed friend on average, and students meet 74.44 distinct schoolmates over the year against 72.31 under simulated random seating (closed-form expectation 72.29).

Contents

1	Problem setting	2
2	Synthetic cohort	2
3	Screens and feasibility certificate	2
4	The seating solver	3
4.1	Weights and costs	4
4.2	Stage 1: pod greedy	4
4.3	Stage 2: swap repair	4
4.4	Closures	4
4.5	Stage 3: simulated annealing	4
4.6	Stage 4: CP-SAT	5
4.7	Baseline, anchors, history	5
5	Statistics, leakage and fairness	5
6	Validation, determinism and cost	5
7	Results	6
A	Trace file schema	6
B	Reproduction	7

*Code and traces: <https://github.com/IsaacLin247/lunch-seating-visual>. Interactive site: <https://isaaclin247.github.io/lunch-seating-visual/>. The companion article (`article/article.tex`) contains the general problem statement, proofs and the case-study analysis; this report documents the implementation stage by stage.

1 Problem setting

One lunch period seats $N = 257$ students ($n_{11} = 132$ juniors, $n_{12} = 125$ seniors) at 40 tables, 17 seating 7 and 23 seating 6, so every seat is filled. Tables are reassigned every two weeks, 16 rotations a year, alternating a *mixed* state (any table) and a *same-grade* state (juniors with juniors, seniors with seniors). `table_layout(state, n11, n12)` computes the layout for any $N \in [240, 280]$: $n_7 = N - 240$ seven-seat tables come first; in the same-grade state the juniors take the largest feasible number of seven-seat tables with $7x_7 + 6x_6 = n_{11}$ and the seniors the rest ($12 \times 7 + 8 \times 6$ and $5 \times 7 + 15 \times 6$ for the default cohort), ordered junior sevens, senior sevens, junior sixes, senior sixes; impossible sizes raise an error instead of returning a malformed layout.

Mechanism. Each student confidentially submits at least four names or none, with at least two names in their own grade. Write L_i for the submitted list ($|L_i| \leq 8$) and $L_i^{(r)}$ for the *effective* list of rotation r (L_i in mixed rotations, same-grade names only in same-grade rotations). Every rotation the solver must seat each submitter with at least one effective listed peer. Among such seatings it prefers exactly one listed peer per student and avoids pairs who have already shared a table.

Notation. $x_{i,t}$ indicates that i sits at table t ; s_i is the number of i 's effective listed peers at i 's table; m_{ab} counts earlier rotations in which a and b shared a table while neither listed the other; α_{ab} counts earlier rotations in which they shared a table while one listed the other. Both histories are updated after every rotation from the lists in force at the time.

2 Synthetic cohort

Students are S001–S257; no real data is used. `make_cohort` in `sim/generator.py` draws popularity $\pi_i \sim \text{LogNormal}(0, 0.6^2)$, then $K = 8$ directed friendships per student in a random order over students: each open slot is filled by a distinct $j \neq i$ with probability proportional to π_j times 9 within grade and 1 across, and each new arc $i \rightarrow j$ is reciprocated with probability p if j has an open slot; p is chosen from a fixed grid so the measured reciprocity is nearest 0.5 (measured 0.490; within-grade share 0.904). Latent groups (μ , ω , group sizes 6–12, optional cross-grade groups) are available and are off in the committed traces (Section 2.2 of the article). Six mid-popularity juniors are planted as mutual true friends and play the coalition.

Submitted lists. Honest students submit their true friends. `apply_rules` treats a list with fewer than four names, or fewer than two same-grade names, as *no submission* (policy `none`); the number of students so excluded is recorded (0 in the committed cohort, 0 non-submitters). A `pad` policy that fills such lists with sampled acquaintances exists for modelling students who name acquaintances to qualify and is not used. The coalition's lists per scenario: a one-name cycle (`k1`); the 4-of-5 *omission star* (`min4`: c_0 omits c_1 , everyone else omits c_0); the earlier two-3-cycles wiring (`min4-3cycles`, kept for comparison); and the *stratified* attack (five members list the next two around a 5-cycle plus two distinct mid-popularity seniors each, the sixth lists four of the five).

3 Screens and feasibility certificate

`sim/screens.py` runs on the submitted lists before rotation 1, once per rotation state, on that state's effective lists.

Definitions. A set A is *closed* when every submitter in A has a listed peer inside A (non-submitters impose nothing). Candidates are the list-closures of submitters (sets closed under following listed names, hence self-contained) of size at most 64. A candidate with at least two

Algorithm 1 One rotation

```
1: raise InfeasibleInputError if a submitter has no eligible listed peer
2:  $\sigma \leftarrow \text{PODGREEDY}$  ▷ Stage 1
3:  $\text{SWAPREPAIR}(\sigma)$  ▷ Stage 2
4:  $\sigma \leftarrow \text{ANNEAL}(\sigma, 300\,000)$  ▷ Stage 3, full-year history
5: for up to 6 attempts while a submitter is friendless do
6:    $\text{SWAPREPAIR}(\sigma); \sigma \leftarrow \text{ANNEAL}(\sigma, 150\,000)$ 
7: end for
8:  $\sigma' \leftarrow \text{CPSAT}(\text{hint} = \sigma)$  ▷ Stage 4, previous-rotation pairs
9: if  $\sigma'$  found, has no friendless submitter, and  $C(\sigma') \leq C(\sigma)$  then  $\sigma \leftarrow \sigma'$ 
10: end if
11: if a submitter is still friendless then
12:    $\sigma' \leftarrow \text{CPSAT}(\text{hint} = \sigma)$  with six times the budget; keep only if it satisfies everyone
13: end if
14: raise RuntimeError if a submitter is still friendless; return  $\sigma$ 
```

submitters is *coercive* when it admits no partition into two closed parts that both contain a submitter; every feasible seating then puts all its submitters at one table.

Coercion test. Exact: for candidates of size at most 20 by enumerating the $2^{|S|-1}$ subsets containing a fixed submitter with bitmask closedness checks; larger candidates pass immediately when every submitter has at least three listed peers among the candidate’s submitters (Thomassen’s theorem gives two disjoint cycles that extend to two closed parts), and are otherwise decided exactly by a CP-SAT model with one Boolean side variable per member and one clause per submitter (a 10 s budget; a timeout would be reported as unverified and flagged).

Demand test. For a candidate C with n_1 submitters and n_0 non-submitters and followers $F(C)$ (submitters outside C whose entire effective list lies in C), at most $n_0 + \lfloor n_1/2 \rfloor$ tables can hold members of $C \cup F(C)$; if $|C| + |F(C)|$ exceeds the seats of that many largest admissible tables the instance is infeasible and the core with its followers is flagged.

Report. Per state: candidates with verdicts and splits, coercion flags, demand flags, submitters with an empty effective list. Overall: flagged students (submitters of coercive sets, cores of over-demanded sets), returned students (flagged plus followers), minimum-list and same-grade violations. The older insular and boundary detectors remain as diagnostic functions and are not used for flagging: a cluster with outward names is not forced anywhere, and flagging such clusters produced false positives.

Feasibility certificate. Before each year, `feasibility_certificate` solves the hard constraints alone with CP-SAT for both states (deterministic budget 20) and records `OPTIMAL`, `INFEASIBLE` or `UNKNOWN`; an infeasible instance stops the run with the certificate. Committed traces: `OPTIMAL / OPTIMAL` in 5.0 s together. The K_5 -plus-ten-followers instance is proved infeasible in a few seconds in both states.

4 The seating solver

One rotation is solved by `solve_rotation` in `sim/solver.py` given lists, state, the two history matrices and the previous rotation’s tables (Algorithm 1).

4.1 Weights and costs

A pair is *listed* if either lists the other. The pair weight is $W_{ab} = 5m_{ab} + 2\alpha_{ab}$ (scaled by 10 relative to the energy $-\frac{1}{10}\sum(5m + 2\alpha)$), so with fixed lists an incidental pair that has met m times weighs $5m$ and a listed pair $2m$, the policy “incidental repeats cost more than anchor repeats”. If a single history matrix of all co-seatings is supplied, it is split by today’s listed relation, which is identical when lists never change. Per-table cost is $\sum_i \text{pen}_i(s_i) + \sum_{\{a,b\}} W_{ab}$ with $\text{pen}_i(0) = 10\,000$ for a submitter and $\text{pen}_i(s) = 10 \max(0, s - 1)$ otherwise; total cost C is the negative of the energy up to a constant, in integers. The state caches per table the cost and the number of friendless submitters; every move reports ΔC and $\Delta\nu$ (change in friendless count) exactly by re-costing the two affected tables.

4.2 Stage 1: pod greedy

Students are processed in increasing order of effective list size (ties random). Each is paired with an unmatched listed peer who lists them back if possible, then by smallest history and shortest list; a peer who does not list back and has strictly fewer options is never taken. The rest become singleton pods. Needy students (no listed peer inside their pod) are processed in increasing order of list size and merge their pod with a listed peer’s pod when the merged size fits the smallest table, choosing by extra familiar faces created, history and size; this assembles chains before other students take the seats beside chain members. Pods are placed largest first into the admissible table with enough seats that minimises history plus one unit per list relation with occupants, ties toward emptier tables; a pod that fits nowhere is split into singletons.

4.3 Stage 2: swap repair

For up to 20 rounds each friendless submitter i tries the single swap with every non-anchor member of every table holding one of i ’s listed peers and applies the cheapest swap after which i has an anchor and the set of friendless submitters is a strict subset of what it was (**best_rescue_swap**); a swap that rescues i by stranding someone else is rejected. Failing that, **kick_and_repair**(i, j) forces the closure of i onto listed peer j ’s table, evicts an equally sized set of j ’s tablemates outside j ’s closure (longest lists first), re-seats evicted or stranded students with their own rescue swaps, and keeps the compound only if the total cost fell and nobody who was satisfied before is friendless afterwards, otherwise restoring the state in place (callers hold aliases to its lists). Repair can stop short on hard instances; the later stages take over.

4.4 Closures

The closure of a set G within its table T is grown until stable by two rules: a submitter of $T \setminus G$ all of whose anchors in T lie in G joins G ; a submitter in G with no listed peer inside G pulls in one of its anchors in T at random. Closures larger than 7 are abandoned. When two closures differ in size the smaller is padded with a random tablemate and *re-closed* (up to four times); if sizes still differ the move is skipped.

4.5 Stage 3: simulated annealing

300 000 iterations of Metropolis acceptance on C with T geometric from 25 to 0.2 in scaled units ($2.5 \rightarrow 0.02$ in energy units). Move mix: probability 1/4 raw single swap with a uniform same-grade-class partner; 1/4 peer-directed single swap (partner drawn from the table of one of a ’s listed peers); otherwise a closure swap with re-closed padding; while $\nu > 0$ every 8th iteration is a targeted move from a friendless submitter toward one of its peers, every 64th of those a kick-and-repair. *Every* candidate move is rejected if $\Delta\nu > 0$, so once $\nu = 0$ the search never leaves the feasible region (verified by an instrumented test); before that, violations may be transferred, which is how chains get assembled. The best assignment under the lexicographic order (friendless

count, cost) is returned. The retry loop of Algorithm 1 keys on the actual friendless count, not on a cost threshold.

4.6 Stage 4: CP-SAT

The model (`_base_model`) has $x_{i,t}$ over allowed tables, exactly-one and capacity constraints, and for every submitter $y_{i,t} \leq x_{i,t}$, $y_{i,t} \leq \sum_{j \in L_i^{(r)}} x_{j,t}$, $\sum_t y_{i,t} \geq 1$; a submitter with an empty effective list raises `InfeasibleInputError` rather than being skipped. `cpsat_polish` adds z_i with $\sum_j x_{j,t} \leq 1 + (\ell_i - 1)z_i + \ell_i(1 - x_{i,t})$ for all t , repeat indicators $r_{ab} \vee \neg x_{a,t} \vee \neg x_{b,t}$ for previous-rotation pairs with $W_{ab} > 0$, objective $300 \sum_i z_i + \sum 100 W_{ab} r_{ab}$, and hints for every variable. Solver: deterministic interleaved search, 8 workers, deterministic-time budget 3.5; with 16 workers on the 8-core development machine repeated solves were not identical, so 8 is used throughout. The polished seating is accepted only if it satisfies everyone and $C(\sigma') \leq C(\sigma)$ on the full history. Timing records greedy, repair, anneal, CP-SAT, fallback and total; the fallback solve is inside the total.

4.7 Baseline, anchors, history

The baseline is a uniformly random seating respecting the state, drawn from a generator seeded independently per rotation. Anchors recorded in the trace are the listed peer at the table with the fewest previous co-seatings. After both seatings, α_{ab} or m_{ab} is incremented for each co-seated pair according to whether the pair is listed; a separate total history serves the baseline and the anchor choice.

5 Statistics, leakage and fairness

Per rotation: percentages of submitters with $s_i \geq 1, = 1, \geq 2$ for both seatings; mean distinct schoolmates met so far; repeat pairs (pairs seated together who had met before); cost breakdown; accepted phase; solve and fallback time; coalition clusters, largest cluster, member-weighted mean cluster ($\sum_c |c|^2/6$), pattern and intactness. Per trace: closed-form expectations for the random baseline (`sim/baseline.py`: pair co-seating probabilities per state, expected distinct schoolmates for the schedule and for an all-mixed year, expected friend coverage per state from the hypergeometric probability that none of a student’s eligible names lands at their table); the leakage report (`sim/privacy.py`: for every submitter, the minimum hitting set of their 16 tablemate sets and the names every hitting set of size at most 8 must contain, scored against the true lists: 613 forced entries, 613 correct, 243 students, 0 fully determined); the distribution of distinct schoolmates met (min 64, quartiles 73.0 and 77.0, max 82) and its means for the lowest and highest in-degree quartiles (75.78 vs 72.48); the submission report; library versions and source commit. The summary lists the rotations in which the proposed seating had more repeat pairs than the random one (3, 5, 7) and, for coalitions, intact rotations by state and the pattern histogram.

6 Validation, determinism and cost

`validate_trace` asserts exact capacities in both seatings of every rotation, every student exactly once, alternating states, no cross-grade table in same-grade rotations, an eligible listed peer at every submitter’s table with the recorded anchor among them, and a reported ≥ 1 percentage of 100. The pytest suite (unit, screen, attack, baseline/leakage, trace and site tests) re-checks these on the committed files and reproduces the audit’s counterexamples: the same-grade capture is flagged only on the effective graph, the K_5 -plus-followers input is over-demanded and proved infeasible, non-submitter sinks and external anchors are not flagged, odd step-two circulants are flagged, repair never strands a satisfied student, the annealer never leaves the feasible region,

Table 1: Headline statistics per scenario (seed 7, 16 rotations). “ ≥ 1 ” and “exactly one” are percentages of submitters; “met” is the mean number of distinct schoolmates a student has shared a table with by rotation 16; cluster statistics describe the six coalition members (mean = $\sum_c |c|^2/6$).

Scenario	≥ 1 (min)	exactly one		met	met, intact	mean
		mean	min		random	cluster
Honest	100.0	99.68	98.4	74.44	72.31	–
Coalition, no defenses	100.0	99.59	98.4	72.78	72.31	16/16
Coalition, min-4 rule (star)	100.0	98.30	97.3	74.00	72.31	0/16
Coalition, same-grade attack	100.0	98.69	96.9	73.33	72.31	8/16
Coalition, screens on	100.0	98.30	97.3	74.00	72.31	0/16

padded groups are re-closed, empty effective lists raise, layouts generalise and invalid sizes raise, and the leakage probe’s deductions are all correct.

Determinism: seeded cohort and local search, seeded baseline, CP-SAT in deterministic interleaved mode with 8 workers, fixed iteration counts, pinned versions (Python 3.14.7, NumPy 2.5.2, OR-Tools 9.15.6755), and version/commit metadata in every trace. Re-exporting a scenario reproduced the committed file byte for byte apart from timestamps and timings. The export of five scenarios took 701 s of solver time; per rotation 8.8 s on average, 10.0 s at most (Stage 3 about 4–5 s, Stage 4 about 3–7 s including model construction).

7 Results

Tables 1–6 are generated from the committed traces and `results/experiments.json`. The honest cohort satisfies every submitter in every rotation; 99.68% have exactly one listed friend on average; 74.44 distinct schoolmates against 72.31 random (closed form 72.29). The one-name cycle is intact 16/16; the omission star is intact 0/16 with mean cluster 3.02 (structural bound 3, the best of all 4-of-5 wirings); the stratified attack is intact in 8 of 8 same-grade rotations and 0 of 8 mixed ones and is flagged by the screen on the same-grade lists (`{S015, S030, S055, S110, S115}`; `{S015, S030, S055, S110, S115, S121}`), returning all 6 colluders and no honest student; the adversarial resubmission yields mean cluster 3.02. Across – independent cohorts exactly-one averages –% and the difference in distinct schoolmates against the paired random baseline ranges from – to –.

A Trace file schema

```
{ "config": { scenario, seed, n, n11, n12, K, rotations, firstState, tableCapacities
  [40],
    sameGradeTableGrade[40], rules, coalitionMode, coalition[], weights,
    network,
    submission, solver, feasibility{mixed,same}, baseline, versions, screen?,
    yearSolveSeconds },
  "students": [ { "id": "S001", "grade": 11}, ... ],
  "listed": [ ["S014", ...], ... ], // submitted lists actually solved
  "listedInitial": [...], // screens-on scenario: the flagged submission
  "hero": "S123",
  "rotations": [ { idx, state, tables, tablesRandom, anchors, anchorsRandom, stats,
    coalition?, coalitionRandom? }, ... ],
  "fairness": { distinctMet{min,q1,median,q3,max,mean}, lowestInDegreeQuartile,
    highestInDegreeQuartile },
  "leakage": { kMax, rotationsObserved, forcedEdges, forcedEdgesCorrect,
    studentsWithForcedEdge,
    fullyDeterminedStudents, submittedEdges, fractionOfEdgesForced },
```

Table 2: Honest scenario, rotation by rotation. “2+” counts submitters with two or more listed peers at their table; repeat pairs are pairs seated together who had already shared a table this year (proposed / random); “phase” is the pipeline stage whose seating was kept; time is the wall-clock solve time of the rotation including any fallback solve.

rot.	state	≥ 1	exactly one	2+	met	met, rnd	repeats (prop./rnd)	phase	time (s)
1	mixed	100	100.0	0	5.5	5.5	0 / 0	cpsat	6.3
2	same	100	100.0	0	10.9	10.8	4 / 17	cpsat	8.0
3	mixed	100	100.0	0	16.1	16.1	27 / 20	cpsat	8.3
4	same	100	100.0	0	21.2	21.1	56 / 66	anneal	8.7
5	mixed	100	100.0	0	26.2	26.1	59 / 47	anneal	8.4
6	same	100	100.0	0	31.1	30.8	71 / 99	anneal	8.5
7	mixed	100	100.0	0	35.9	35.6	86 / 85	anneal	8.2
8	same	100	99.2	2	40.5	40.0	112 / 135	anneal	8.5
9	mixed	100	100.0	0	45.1	44.7	103 / 108	anneal	8.6
10	same	100	100.0	0	49.6	48.8	134 / 179	anneal	7.9
11	mixed	100	99.6	1	54.0	53.1	124 / 138	anneal	8.3
12	same	100	98.4	4	58.2	56.9	175 / 218	anneal	8.6
13	mixed	100	100.0	0	62.4	61.0	152 / 177	anneal	8.5
14	same	100	98.4	4	66.3	64.8	201 / 214	anneal	8.5
15	mixed	100	100.0	0	70.7	68.8	145 / 180	anneal	9.1
16	same	100	99.2	2	74.4	72.3	217 / 257	anneal	8.7

```
"summary": { pctGe1Min, pctExactly1Mean, pctExactly1Min, pctGe1RandomMean,
  meanDistinctMetFinal,
    meanDistinctMetFinalRandom, repeatsWorseThanRandomRotations, phases,
      maxSolveSeconds,
        coalitionIntactRotations?, coalitionIntactByState?, coalitionAvgCluster?,
          coalitionPatterns? } }
```

B Reproduction

```
python -m venv .venv && . .venv/bin/activate && pip install -r requirements.txt
python sim/export_traces.py # five scenarios, ~15 min, deterministic
python sim/experiments.py --seeds 10 # robustness runs
pytest
python paper/make_tables.py && cd paper && latexmk -pdf methodology.tex
```

Table 3: Coalition outcome per rotation under each defense level: partition pattern of the six members across tables (“6” = all at one table) and the member-weighted mean cluster size.

rot.	state	no defenses		min-4 (star)		same-grade attack		screens on	
		pattern	mean	pattern	mean	pattern	mean	pattern	mean
1	mixed	6	6.00	3+3	3.00	2+1+1+1+1	1.33	3+3	3.00
2	same	6	6.00	3+3	3.00	6	6.00	3+3	3.00
3	mixed	6	6.00	3+3	3.00	2+2+1+1	1.67	3+3	3.00
4	same	6	6.00	3+3	3.00	6	6.00	3+3	3.00
5	mixed	6	6.00	4+2	3.33	2+2+1+1	1.67	4+2	3.33
6	same	6	6.00	3+3	3.00	6	6.00	3+3	3.00
7	mixed	6	6.00	3+3	3.00	2+2+1+1	1.67	3+3	3.00
8	same	6	6.00	3+3	3.00	6	6.00	3+3	3.00
9	mixed	6	6.00	3+3	3.00	2+1+1+1+1	1.33	3+3	3.00
10	same	6	6.00	3+3	3.00	6	6.00	3+3	3.00
11	mixed	6	6.00	3+3	3.00	2+1+1+1+1	1.33	3+3	3.00
12	same	6	6.00	3+3	3.00	6	6.00	3+3	3.00
13	mixed	6	6.00	3+3	3.00	2+1+1+1+1	1.33	3+3	3.00
14	same	6	6.00	3+3	3.00	6	6.00	3+3	3.00
15	mixed	6	6.00	3+3	3.00	3+2+1	2.33	3+3	3.00
16	same	6	6.00	3+3	3.00	6	6.00	3+3	3.00

Table 4: Screen on the coalition’s initial submission (same-grade attack): candidate closed sets on each rotation state’s effective lists and the verdict.

state	candidate closed set S	verdict	closed split found
mixed	(no closed set of at most 64 students)	–	–
same	S015, S030, S055, S110, S115	flagged	none
same	S015, S030, S055, S110, S115, S121	flagged	none

Table 5: Structural cohesion bounds for six-member wirings: the minimum, over all partitions of the six into closed parts, of the member-weighted mean cluster size and of the largest cluster. These bounds hold for every feasible seating regardless of solver, seed or history.

wiring	min mean cluster	min largest cluster	passes screen
one name each, cycle (k=1)	6.0	6	no (coercive)
4-of-5, omitted names form two 3-cycles	2.0	2	yes
4-of-5, omission star (nobody lists member 1)	3.0	3	yes
best of all 5^5 4-of-5 wirings (exhaustive)	3.0	3	yes

Table 6: Parameters used for the committed traces.

group	parameter	value
cohort	students (grade 11 / 12)	257 (132 / 125)
cohort	true friends per student K	8
cohort	popularity σ (lognormal)	0.6
cohort	within-grade weight	9.0
cohort	reciprocity target / measured / p	0.5 / 0.490 / 0.60
cohort	measured within-grade fraction	0.904
cohort	latent groups μ, ω	0.0, 0.0
rules	minimum list / same-grade names / short lists	4 / 2 / no submission
tables	capacities	$17 \times 7, 23 \times 6$
rotations	count / first state	16 / mixed
stage 3	satisfied submitter	1000.0
stage 3	extra listed peer	1.0
stage 3	repeat per prior meeting: incidental / listed	0.5 / 0.2
stage 3	temperature $T_0 \rightarrow T_1$	$2.5 \rightarrow 0.02$
stage 3	iterations	300,000
stage 4	weight per student with ≥ 2 peers	300
stage 4	repeat scale	100
stage 4	deterministic time budget / workers	3.5 / 8
certificate	feasibility budget (deterministic time)	20.0
run	seed / versions	7 / 3.14.7, 2.5.2, 9.15.6755